AIORI-3 HACKATHON 2026

PURPOSE–BASED CONSENT RECORDING & WITHDRAWAL PROTOTYPE

Problem Statement ID - PS002

Theme - Cybersecurity

Team Name - SPARK

Team Members - Pradeep Yadav, Yash Chandra

Faculty Guide - Dr. Tejasvi Khanna

Table of Contents

01	**Title Slide**	Project title (ConsentGuard), team details and hackathon.
02	**Table of Contents**	Overview of the presentation.
03	**Problem Statement & Objectives**	Problem background, key issues and project objectives.
04	**Proposed Solution & Notice-Change Policy**	Consent workflow, notice versions, withdrawal and re-consent.
05	**System Architecture & Pseudocode**	Frontend, backend, mock data-sharing API, database and enforcement logic.
06	**Demonstration Flow**	Step-by-step working of the prototype.
07	**Lifecycle & Enforcement Test Cases**	12 test cases for consent lifecycle, notice change and access control.
08	**Optimization & Timeline**	Improvements, official hackathon timeline and future scope.
09	**Conclusion, Audit Evidence & References**	Key takeaways, sample audit records, CSV/JSON export and references.

PROBLEM STATEMENT & OBJECTIVES

Addressing the lack of purpose-based consent control in digital platforms

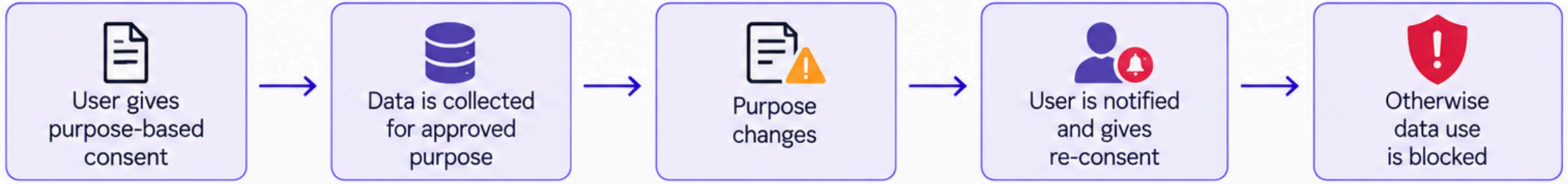
PROBLEM STATEMENT

Users give consent to apps and websites without fully understanding how their data will be used. After giving consent, they have limited control, find it difficult to withdraw consent, and are often not clearly informed when the purpose of data usage changes. This can lead to misuse of personal data, lack of transparency, and loss of user trust.

OBJECTIVES

1. Enable purpose-based consent for data usage.
2. Provide a simple and clear way to withdraw consent.
3. Notify users when the purpose of data usage changes and require re-consent.
4. Maintain a transparent audit log of all consent activities.
5. Improve user trust and support data protection best practices.

IDEA IN A SINGLE FLOW



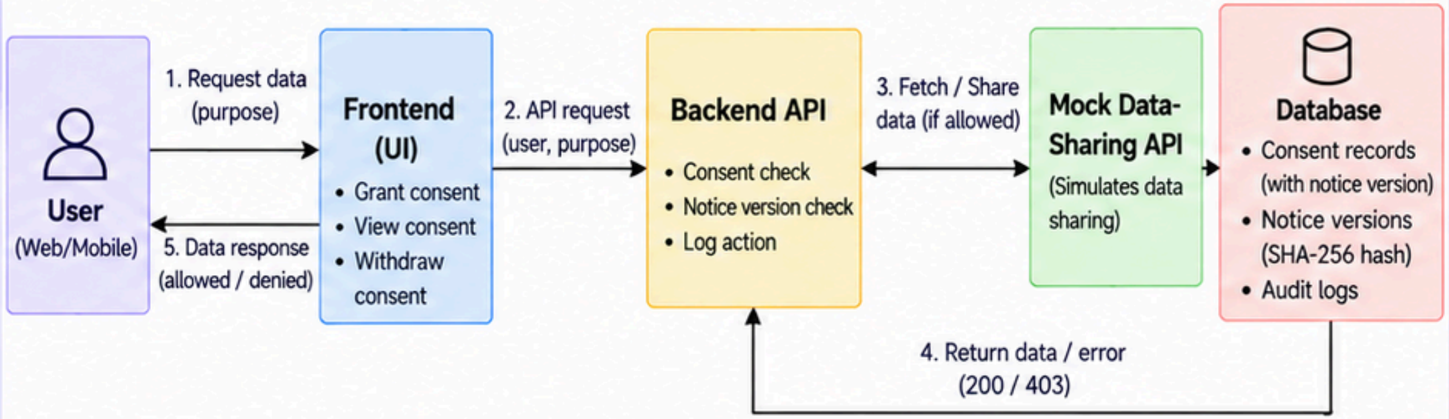
Proposed Solution & Notice-Change Policy

A consent-based data access system with technical enforcement and audit evidence

Proposed Solution

- Users give consent for a specific purpose (Analytics / Marketing / Service Improvement) and for a defined duration.
- Backend checks the consent status and notice version before allowing any data access.
- All consent actions and data access requests are stored in an audit log.
- A mock data-sharing API is used to simulate sharing data only when consent is valid.
- Users can view, withdraw and re-consent anytime through a simple interface.

System Flow (High Level)



Notice-Change Policy

- When the notice version changes, all existing consents become invalid. Users must re-consent to continue accessing data.
- Each notice version has a unique SHA-256 hash (e.g., v1, v2 ...).
- Old consents (with older notice hash) are not valid after a change.
- If a user requests data with an old consent, the system will deny access (403).
- User can grant new consent with the latest notice version to continue.

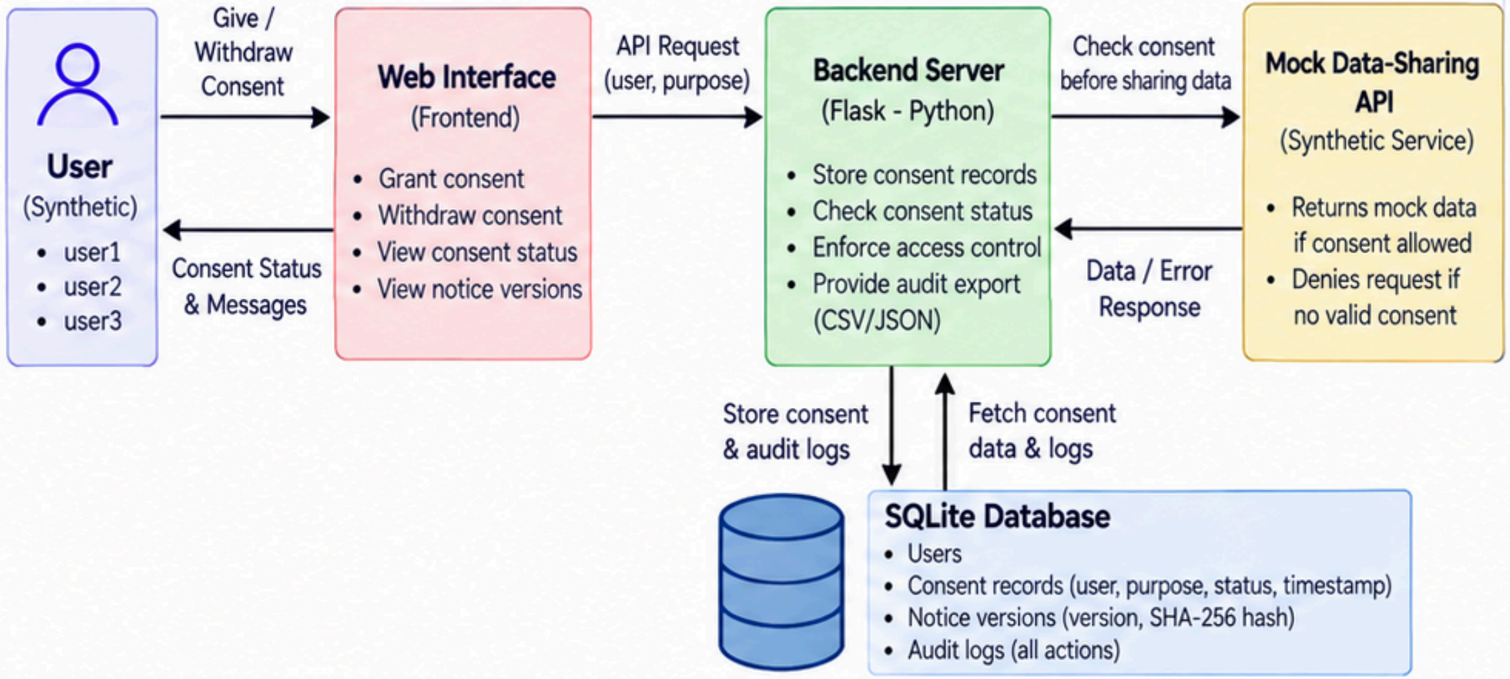
Supported Purposes & Consent Types

Purpose	Data Examples (Mock)	Consent Type	Retention / Duration
Analytics	Usage data, feature usage	Text	e.g., 6 months
Marketing	Email, preferences	Text	e.g., 6 months
Service Improvement	App performance, error logs	Text	e.g., 6 months

System Architecture & Pseudocode

Architecture of ConsentGuard with enforcement through a mock data-sharing API

System Architecture



Key Components

- **Frontend:** Simple web interface to grant, withdraw and view consent.
- **Backend:** Flask server that stores consent, checks current consent status and enforces access control.
- **Mock Data-Sharing API:** Returns synthetic data only if valid consent exists for the requested purpose and latest notice version.
- **Database:** SQLite to store users, consent records, notice versions (with SHA-256 hash) and audit logs.
- **Audit Export:** Export logs in CSV or JSON format for technical consent evidence.

Pseudocode (Enforcement Logic)

```
def request_data(user_id, purpose):
    # Get current notice version and hash
    notice = get_current_notice(purpose)
    current_version = notice.version
    current_hash = notice.hash

    # Get latest consent for user and purpose
    consent = get_consent(user_id, purpose)

    # Check consent status and notice version
    if consent is None:
        return "403 - No consent found"
    if consent.status != "granted":
        return "403 - Consent withdrawn"
    if consent.notice_version != current_version:
        return "403 - Consent invalid (notice changed)"

    # All checks passed, call mock API
    data = get_mock_data(purpose)
    return "200 - Allowed", data
```

Mock API Example

1. Allowed Response

GET /api/data?user_id=user1&purpose=analytics

Response (200):

```
{
    "status": "success",
    "data": {
        "visits": 120,
        "location": "IN"
    }
}
```

2. Denied Response

GET /api/data?user_id=user1&purpose=marketing

Response (403):

```
{
    "status": "error",
    "code": 403,
    "message": "No consent found"
}
```

DEMONSTRATION FLOW

Step-by-step working of the prototype

1. **User Registration / Login**
   User creates an account to securely manage their consents.

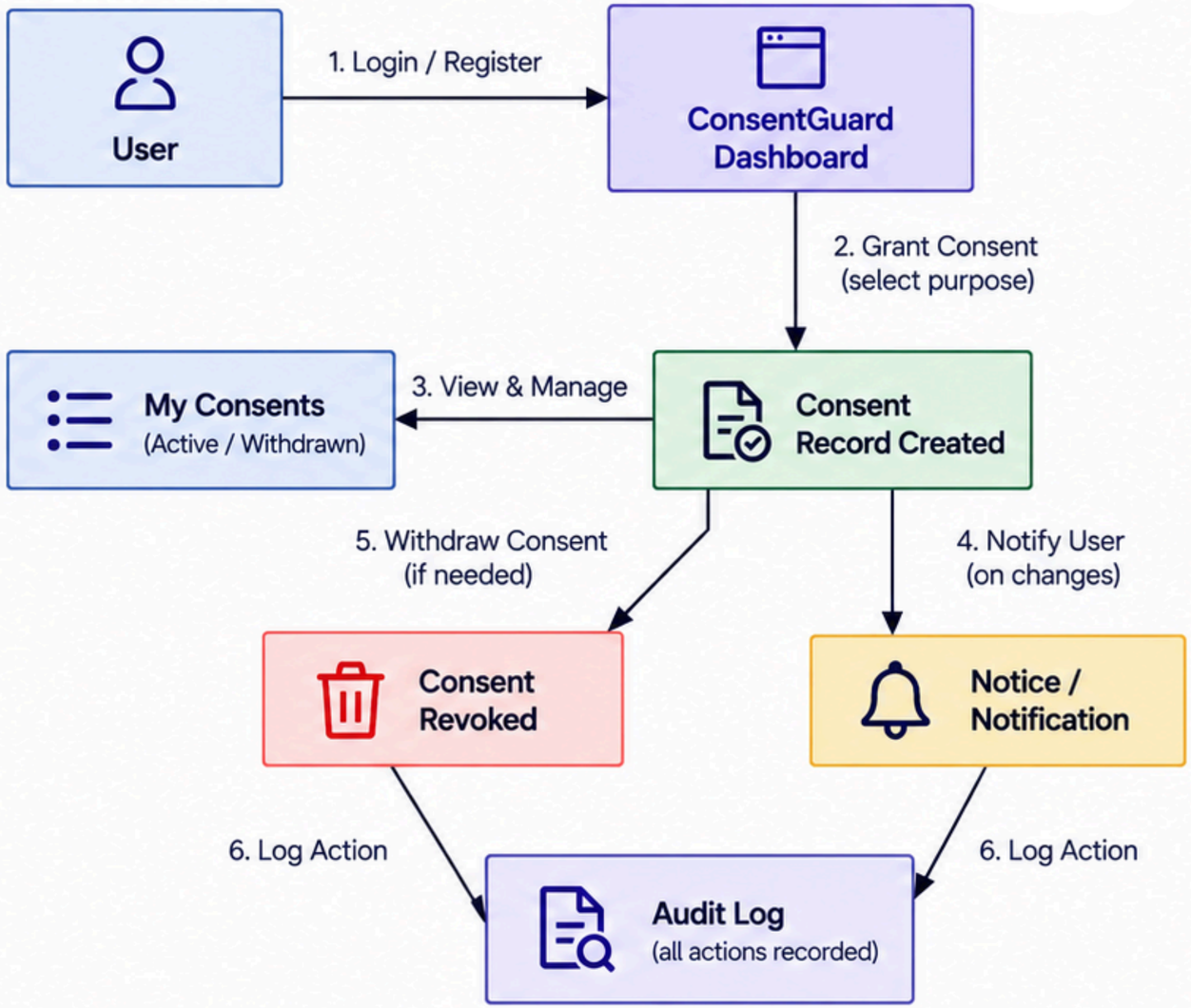
2. **Grant Consent**
   User gives consent for a specific purpose by selecting purpose and duration.

3. **View & Manage Consents**
   User can view all their active consents and check details like purpose, date and status.

4. **Withdraw Consent**
   User can revoke a previously given consent with a single click.

5. **Notice Change & Re-Consent**
   User receives a notification if there is any change in purpose or policy and can choose to continue or withdraw.

6. **Audit Log**
   All consent actions (grant, withdrawal, changes) are logged for transparency and accountability.



Lifecycle & Enforcement Test Cases

12 test cases covering grant, withdrawal, notice change and access control (Expected Results)

#	Test Case	User	Purpose	Precondition	Action (API / UI)	Expected Result	Actual Result	Status
1	Grant consent (Analytics)	user1	Analytics	No previous consent	Grant consent (v1)	Consent recorded (granted)	Not tested	Not tested
2	Access with consent (Analytics)	user1	Analytics	Consent granted (v1)	GET /api/data (analytics)	Allowed (200)	Not tested	Not tested
3	Access without consent	user2	Marketing	No consent	GET /api/data (marketing)	Denied (403)	Not tested	Not tested
4	Withdraw consent	user1	Analytics	Consent granted (v1)	Withdraw consent	Consent marked withdrawn	Not tested	Not tested
5	Access after withdrawal	user1	Analytics	Consent withdrawn	GET /api/data (analytics)	Denied (403)	Not tested	Not tested
6	Grant consent (Marketing)	user2	Marketing	No previous consent	Grant consent (v1)	Consent recorded (granted)	Not tested	Not tested
7	Access with consent (Marketing)	user2	Marketing	Consent granted (v1)	GET /api/data (marketing)	Allowed (200)	Not tested	Not tested
8	Notice change (invalidates consent)	user1	Analytics	Consent granted (v1)	Update notice to v2	Old consent becomes invalid	Not tested	Not tested
9	Access with old consent after notice change	user1	Analytics	Consent from v1, notice now v2	GET /api/data (analytics)	Denied (403)	Not tested	Not tested
10	Re-consent after notice change	user1	Analytics	Notice version v2	Grant consent (v2)	Consent recorded (granted)	Not tested	Not tested
11	Access after re-consent	user1	Analytics	Consent granted (v2)	GET /api/data (analytics)	Allowed (200)	Not tested	Not tested
12	Multiple purpose check	user3	Service Improvement	No consent	GET /api/data (service)	Denied (403)	Not tested	Not tested

Notes

- Three purposes used: Analytics, Marketing and Service Improvement.
- Notice change invalidates previous consent and user must re-consent.
- Access is allowed only if consent exists for the requested purpose and the notice version matches.
- All test cases are planned and not tested yet. They will be executed in the next iteration.

Optimization & Timeline

Planned improvements and official hackathon timeline

Optimizations / Future Improvements

- Add user authentication (e.g., email/OTP) for real-world usage.
- Improve UI with better consent management dashboard.
- Support granular data selection (e.g., specific data fields).
- Add retention period and auto-expiry of consent.
- Extend to role-based access (e.g., admin, developer).
- Deploy on cloud for wider access.
- Add support for multiple notice versions with history view.
- Integrate with real data-sharing APIs in future (beyond mock).

Scope for Hackathon (Current Version)

- Web interface for users to grant, view and withdraw consent.
- Backend enforcement logic based on consent status and notice version.
- Mock data-sharing API to simulate data access.
- Audit logging of all actions.
- Support for 3 purposes (Analytics, Marketing, Service Improvement).
- Notice change handling (old consent becomes invalid).
- Export audit logs (CSV and JSON).
- 12 test cases to verify lifecycle and access control.

Project Timeline (Official Dates)

Phase	Dates	Key Activities	Deliverables
Level 1 Submistion	12 – 14 Oct	• Set up GitHub repository • Prepare problem statement PDF • Record and submit short video	Repo link, PDF and video submitted on hackathon portal
Mentoring & Build Phase	16 – 31 Oct	• Attend mentoring sessions • Build and test the prototype • Run 12 test cases • Generate audit export (CSV/JSON)	• Working prototype • Test results • Audit export file • Updated documentation (if required)
Evaluator Connect	31 Oct – 04 Nov	• Interact with evaluators (if scheduled) • Prepare for demo • Refine based on feedback	• Demo ready • Final submission for evaluation
Regional Round (if selected)	24 – 30 Nov	• Present project in regional round • Respond to questions and feedback	• Final presentation and live demo

Conclusion, Audit Evidence & References

Conclusion

- ConsentGuard provides a simple and practical way to manage purpose-based consent for data access.
- It records user consent, notice version and timestamp, and enforces access control at the backend.
- The mock data-sharing API allows data only if valid consent exists for the requested purpose and latest notice version.
- Withdrawal and notice changes are handled properly by denying unauthorized requests.
- The prototype can be extended to add more purposes, additional security and real-world integrations.

Key Learnings

- Understood the importance of purpose-based consent and notice versioning.
- Learned how to enforce consent at the backend before sharing data.
- Implemented audit logging with proper timestamps and notice hash.
- Gained experience in building a complete workflow with frontend, backend and API.
- Understood the role of transparency and user control in data handling.

Future Scope

- Add more purposes and user roles.
- Use a real database and authentication.
- Integrate with external services and dashboards.
- Improve UI and add analytics for consent history.

Sample Audit Records (From Prototype)

#	Action	User	Purpose	Notice Version	Result	Timestamp
1	Grant	user1	Analytics	v1	Granted	2024-10-16 10:21
2	Data access	user1	Analytics	v1	Allowed (200)	2024-10-16 10:22
3	Withdraw	user1	Analytics	v1	Withdrawn	2024-10-16 11:05
4	Data access	user1	Analytics	v1	Denied (403)	2024-10-16 11:06
5	Notice change	-	-	v2	Notice updated	2024-10-17 09:30
6	Data access	user1	Analytics	v1 (old)	Denied (403)	2024-10-17 09:31

Audit Export (Sample)

CSV (sample)

```
timestamp,user,purpose,action,result,notice_version
2024-10-16 10:21,user1,Analytics,grant,granted,v1
2024-10-16 10:22,user1,Analytics,access,allowed,v1
2024-10-16 11:05,user1,Analytics,withdraw,withdrawn,v1
2024-10-16 11:06,user1,Analytics,access,denied,v1
2024-10-17 09:30,-,-,notice_change,updated,v2
2024-10-17 09:31,user1,Analytics,access,denied,v1
```

JSON (sample)

```
[
  {
    "timestamp": "2024-10-16 10:21",
    "user": "user1",
    "purpose": "Analytics",
    "action": "grant",
    "result": "granted",
    "notice_version": "v1"
  },
  ...
]
```

References

[1] MeitY – Data Protection Framework (PS-002): https://www.meity.gov.in/data-protection-framework
[2] Digital Personal Data Protection Act (DPDP Act), 2023 – Government of India.
[3] GDPR – General Data Protection Regulation, European Union.
[4] NIST Privacy Framework – National Institute of Standards and Technology.
[5] OWASP – Secure Web Application Guidelines.
[6] Research papers and articles on consent management and data privacy.